INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
DISCIPLINA: Padrões de Projeto em Aplicações Web
PROFESSOR: Éder Pereira
ATIVIDADE PRÁTICA: Padrão Estrutural Decorator

Contextualização do Problema

Em um sistema de envio de mensagens corporativas, temos uma implementação básica de envio de texto. No entanto, surgiram novas necessidades operacionais de segurança e otimização de rede, tais como adicionar criptografia para dados sensíveis e compressão de dados para reduzir o tráfego de rede.

Alterar a classe base do emissor ou criar diversas subclasses concretas para cada combinação possível (por exemplo, `EmissorCriptografado`, `EmissorComprimido`, `EmissorCriptografadoEComprimido`) gera uma explosão de classes e um código rígido.

Para solucionar esse problema e permitir a adição de responsabilidades de forma flexível e encadeável, aplicaremos o padrão estrutural **Decorator**. O objetivo do **Decorator** é anexar responsabilidades adicionais a um objeto dinamicamente sem alterar sua estrutura ou afetar outros objetos da mesma classe.

Roteiro de Implementação

1. **Criação do Projeto:**
   - Crie um projeto Java no Eclipse chamado `Decorator`.
2. **Definição da Interface `Emissor`:**
   - Crie a interface `Emissor.java` que estabelece o contrato básico para todos os emissores e decoradores.

```java
public interface Emissor {
    void envia(String mensagem);
}
```

3. **Criação da Classe Concreta (`EmissorBasico`):**
   - Crie a classe `EmissorBasico.java` que implementa a interface `Emissor` com o comportamento padrão de envio de mensagens.

```java
public class EmissorBasico implements Emissor {
    // Comportamento: Exibir no console a mensagem enviada de forma
       simples.
    public void envia(String mensagem) {
        // TODO: Implementar o envio basico de mensagem
    }
}
```

4. **Criação da Classe Abstrata Decorador Base (`EmissorDecorator`):**
   - Crie a classe abstrata `EmissorDecorator.java` que implementa `Emissor` e mantém uma referência encapsulada para um objeto do tipo `Emissor`.

```java
public abstract class EmissorDecorator implements Emissor {
    private Emissor emissor;

    public EmissorDecorator(Emissor emissor) {
```

```
        this.emissor = emissor;
    }

    public Emissor getEmissor() {
        return this.emissor;
    }

    // Assinatura do m todo abstrato a ser redefinido pelos decoradores
       concretos
    public abstract void envia(String mensagem);
}
```

5. **Modelagem dos Decoradores Concretos:**

   - Crie as classes `EmissorDecoratorComCriptografia.java` e `EmissorDecoratorComCompressao.java` herdando de `EmissorDecorator`.
   - Defina os métodos privados auxiliares de transformação e redefina o método `envia(String mensagem)` delegando a execução para o emissor encapsulado.

```
public class EmissorDecoratorComCriptografia extends EmissorDecorator {

    public EmissorDecoratorComCriptografia(Emissor emissor) {
        super(emissor);
    }

    // Comportamento: Processar a mensagem aplicando o m todo privado de
       criptografia e repassar o resultado para o emissor decorado.
    public void envia(String mensagem) {
        // TODO: Criptografar a mensagem e invocar o m todo envia do
           emissor encapsulado
    }

    private String criptografa(String mensagem) {
        // TODO: Implementar a l gica de inversao ou algoritmo de
           criptografia da mensagem
        return null;
    }
}

public class EmissorDecoratorComCompressao extends EmissorDecorator {

    public EmissorDecoratorComCompressao(Emissor emissor) {
        super(emissor);
    }

    // Comportamento: Processar a mensagem aplicando a compress o de
       dados e repassar o resultado para o emissor decorado.
    public void envia(String mensagem) {
        // TODO: Comprimir a mensagem e invocar o m todo envia do emissor
            encapsulado
    }

    private String comprime(String mensagem) {
        // TODO: Implementar a logica de compressao de dados da mensagem (
           ex: uso de DeflaterOutputStream ou manipula  o de bytes)
        return null;
    }
}
```

6. **Testes do Sistema (`TesteEmissorDecorator`):**

   - Crie a classe de testes demonstrando a aplicação individual e a composição encadeada de múltiplos decoradores sobre o emissor básico.

```
public class TesteEmissorDecorator {
    public static void main(String[] args) {
        // TODO: Instanciar um EmissorBasico com decorador de Criptografia e testar o envio
        // TODO: Instanciar um EmissorBasico com decorador de Compressao e testar o envio
        // TODO: Criar um emissor encadeado (Criptografia + Compressao + EmissorBasico) e testar o envio combinado
    }
}
```